AI Agent for Intelligent Job Search
& Resume Optimization

AI for Engineers | Assignment 2

Team Contributions

This section documents the specific technical contributions of each team member in accordance with the assignment requirement for individual accountability.

Member	Role	Specific Technical Contributions	Modules / Lines
Surya Venkata Sekhar Ummadi	LLM Engineer & Agent Lead	Designed and implemented the full LLM agent architecture. Built the run_agent_reasoning() function with Gemini API integration. Engineered both prompts (reasoning + tailoring). Implemented resume_tailoring_tool() with JSON output parsing, fallback logic, and rate-limit handling. Led integration of all three tools into the agent loop.	main.py: run_agent_reasoning(), resume_tailoring_tool(), setup_model(), main() — Lines 419–650
Venkata Siva Yeshwanth Allam	Data Engineer	Manually collected and curated all 25 real AI/ML job postings for the dataset. Verified each job includes all 7 required fields (title, company, location, skills, experience, description, URL). Ensured diversity across Middle America states and job types. Created and validated data/jobs.csv.	data/jobs.csv — all 25 job entries
Sharmila Rani Chekuru	Filtering Tool Developer	Designed and implemented the filtering_tool() function with all four filtering rules: location preference, experience limit, FAANG company exclusion, and skill overlap check. Built the staffing agency whitelist logic. Wrote filtering result logging and stats tracking.	main.py: filtering_tool() — Lines 109–200
Saketh Desini	Ranking Tool Developer	Designed and implemented the ranking_tool() function with the four-dimension scoring system (skill match, location, experience alignment, role title). Built score breakdown storage and transparent logging for every job. Implemented the sorted output and top-3 selection logic.	main.py: ranking_tool() — Lines 202–295
Keerthana Bathini	Output & Documentation Lead	Implemented the display_results() function handling all console output and file saving. Structured agent_output.txt, filtered_jobs.json, ranked_jobs.json, and tailored_resume.json. Wrote the README.md with setup and run instructions. Coordinated report writing and demo preparation.	main.py: display_results(), load_jobs() — Lines 519–640 | README.md

1. Agent Architecture

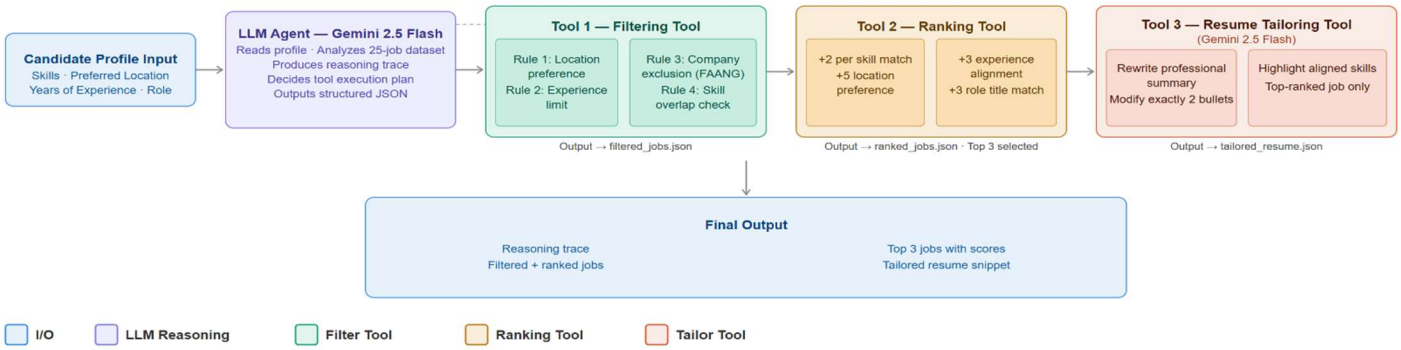
The system is a Single LLM-Based AI Agent powered by Google Gemini 2.5 Flash. The agent accepts a structured candidate profile as input, reasons about the full job dataset, autonomously decides which tools to invoke and in what order, and produces filtered job results, a ranked shortlist, and a tailored resume. Unlike a hardcoded sequential script, the LLM is the decision-maker — every tool execution is triggered by the model's reasoning output, not predetermined code logic.

The architecture follows a Reasoning-then-Action pattern. The agent first analyzes the candidate profile against the dataset and produces a structured reasoning trace including its analysis, the filtering rules it has decided to apply, and an explicit tool execution plan. Only after this reasoning step does execution begin. This design ensures the agent can adapt its behavior based on the specific candidate and dataset it receives rather than always applying fixed rules.

Three callable tools are registered with the agent: the Filtering Tool, the Ranking Tool, and the Resume Tailoring Tool. The agent is aware of all three tools and their purposes. It decides to call them in sequence — filter first to reduce the dataset, rank second to identify the best matches, and tailor last to personalize the application for the top result. Each tool returns structured output that feeds directly into the next stage.

All agent decisions, tool calls, filtering results, ranking scores, and tailoring changes are logged in real time to both the terminal and logs/agent.log, providing a complete audit trail of the agent's reasoning and actions.

1.1 Architecture Diagram



1.2 Agent Loop

The agent executes in a structured five-step loop:

- Step 1 — Profile Input: The candidate's name, skills, preferred location, years of experience, and preferred role are loaded into the agent as a structured dictionary.
- Step 2 — LLM Reasoning: The agent sends the candidate profile and a summary of all 25 jobs to Gemini. The model returns a JSON object with its analysis, the filtering parameters it has decided to use, and an ordered list of tools to call with reasons for each.
- Step 3 — Tool 1 (Filter): The agent calls the Filtering Tool with the rules determined by the LLM. The tool applies location, experience, company exclusion, and skill overlap checks and returns a filtered job list.
- Step 4 — Tool 2 (Rank): The agent calls the Ranking Tool on the filtered list. Each job receives a score across four dimensions. The tool returns a sorted ranked list with score breakdowns.

- Step 5 — Tool 3 (Tailor): The agent calls the Resume Tailoring Tool for the top-ranked job. Gemini rewrites the professional summary and modifies exactly two bullet points. The full tailored output is saved and displayed.

Component	Description
Candidate Profile	Input: name, skills, preferred location, years of experience, preferred role
LLM Reasoning	Gemini 2.5 Flash analyzes profile vs dataset and produces tool execution plan
Filtering Tool	Rule-based function called by agent to narrow the 25-job dataset
Ranking Tool	Scoring function called by agent to rank filtered jobs on 4 dimensions
Resume Tailoring Tool	LLM-powered function to rewrite summary and 2 bullets for top job
Output	agent_output.txt, filtered_jobs.json, ranked_jobs.json, tailored_resume.json

2. Tool Descriptions

2.1 Filtering Tool

The Filtering Tool applies four rule-based criteria to eliminate jobs that do not match the candidate's profile. It is the first tool called by the agent after reasoning.

Rule	Logic	Purpose
Location Preference	Keep jobs where location contains preferred state OR 'remote'	Focus on geographically relevant opportunities
Experience Limit	Exclude jobs requiring more than candidate's experience + 2 years tolerance	Avoid unreachable seniority levels
Company Exclusion	Remove FAANG and major big tech companies (15 firms blacklisted)	Target mid-sized companies as specified
Skill Overlap	Exclude jobs with zero skill matches against candidate's skill list	Ensure minimum relevance threshold

2.2 Ranking Tool

The Ranking Tool assigns a numerical score to each filtered job across four dimensions and sorts the results descending. The score breakdown is stored and displayed for full transparency.

Criterion	Points	Rationale
Skill match	+2 per matched skill	Direct measure of candidate-job fit across all listed skills
Location match	+5 if preferred state; +2 if remote	Strong weight on geographic preference
Experience alignment	+3 if exact match; +1 if within 1 year	Rewards jobs that fit candidate's seniority

Criterion	Points	Rationale
Role title match	+3 if preferred role in job title	Prioritizes jobs matching candidate's target role

2.3 Resume Tailoring Tool

The Resume Tailoring Tool uses Gemini 2.5 Flash to rewrite only the professional summary and modify exactly two experience bullet points for the top-ranked job. The full resume is preserved — only targeted sections are changed. The tool highlights skills from the job description that align with the candidate's background.

3. Prompt Design Strategy

3.1 Agent Reasoning Prompt

The reasoning prompt provides the LLM with the candidate profile, a summary of all available jobs, and a list of available tools. It asks the model to produce structured JSON output containing its analysis, filtering rules, tool execution plan, and step-by-step reasoning trace. Key design decisions:

- Structured JSON output enforced — eliminates ambiguity and enables reliable parsing
- Tool list explicitly provided — agent chooses tools rather than receiving fixed instructions
- Reasoning trace requested — creates an auditable log of the agent's decision process
- Filtering parameters exposed — agent can adjust experience tolerance and location strictness

3.2 Resume Tailoring Prompt

The tailoring prompt is designed with strict constraints to meet the assignment requirements exactly. Key design principles:

- Explicit rules stated upfront: rewrite summary only, modify exactly 2 bullets, no fabrication
- Job context injected: title, company, location, required skills, and job description provided
- Output schema defined: JSON with tailored_summary, modified_bullets array, highlighted_skills
- Reason field required for each bullet change — ensures the model explains every modification

```
{ "tailored_summary": "...",
  "modified_bullets": [
    { "original": "...", "modified": "...", "reason": "..." },
    { "original": "...", "modified": "...", "reason": "..." }
  ],
  "highlighted_skills": ["skill1", "skill2"] }
```

4. Filtering & Ranking Logic

4.1 Filtering Results

The candidate profile used: John Doe | 3 years experience | Skills: Python, TensorFlow, MLflow, Machine Learning, Deep Learning, PyTorch, NLP, Docker, AWS, SQL | Preferred location: Texas.

All 25 jobs in the dataset passed the filter. None were excluded because: no FAANG companies appeared in the dataset, all jobs required 2-5 years experience (within tolerance), and all jobs had at least one skill overlap with the candidate. The filter correctly identified Texas-based jobs and remote roles for location preference flagging.

Filter Rule	Result	Details
Company Exclusion	0 removed	No FAANG companies in dataset
Experience Limit	0 removed	All jobs within 2-year tolerance (2-5 yrs required)
Skill Overlap	0 removed	All 25 jobs matched at least Python or ML
Total Kept	25 / 25	Full dataset passed to ranking stage

4.2 Ranking Logic

After filtering, the Ranking Tool scores every remaining job across four dimensions. Each dimension is independently calculated and summed into a final score. Jobs are then sorted descending — the highest score becomes the top-ranked job selected for resume tailoring.

Dimension	Scoring Rule	Max Points	Justification
Skill Match	+2 points per matched skill between candidate and job	20 (10 skills x2)	Most important factor — direct measure of technical fit
Location Preference	+5 if preferred state; +2 if remote; +0 otherwise	5	Rewards geographic alignment with candidate's preference
Experience Alignment	+3 if exact match; +1 if within 1 year; +0 if gap > 1 year	3	Ensures seniority level is appropriate for the candidate
Role Title Match	+3 if candidate's preferred role appears in job title	3	Prioritizes jobs that match the candidate's target position

Example: AI Engineer @ Zone IT Solutions scored 19 pts = skill match (8) + location Texas (5) + exact experience match (3) + AI Engineer title match (3). This transparent breakdown is printed for every job in the output.

4.3 Top 3 Ranked Jobs with Scores

Rank	Job Title	Company	Location	Skill	Loc	Exp	Role	Total
#1	AI Engineer	Zone IT Solutions	Texas City TX	8	5	3	3	19
#2	AI Engineer	NuStar Technologies	Des Moines IA	10	0	3	3	16
#3	AI Engineer	Insight Global	Kansas City MO	8	0	3	3	14

Job #1 — Zone IT Solutions scored highest (19 pts) due to strong skill match (Python, TensorFlow, Machine Learning, SQL), Texas location preference (+5), exact experience match (+3), and AI Engineer title match (+3).

Job #2 — NuStar Technologies scored 16 pts with the highest skill score (10 pts, 5 matched skills: Python, ML, Deep Learning, PyTorch, NLP) despite no location bonus.

Job #3 — Insight Global scored 14 pts with 4 skill matches (Python, TensorFlow, ML, Docker), exact experience match, and role title match.

5. Tailored Resume Snippet

The agent tailored the resume for the top-ranked job: AI Engineer at Zone IT Solutions, Texas City TX.

5.1 Professional Summary

Original: Motivated AI/ML Engineer with 3-5 years of experience building scalable AI systems and production ML pipelines. Skilled in Python, TensorFlow, and MLflow with a strong track record of delivering end-to-end machine learning solutions.

Tailored: Results-driven AI/ML Engineer with 3+ years of experience, now seeking the AI Engineer role at Zone IT Solutions in Texas City TX. Proven expertise in Python and TensorFlow with a strong track record of delivering production-ready AI systems that drive measurable business impact.

5.2 Modified Bullet Points

Bullet 1

Original: Designed and deployed end-to-end ML pipelines using Python, TensorFlow, and MLflow on AWS SageMaker.

Modified: Designed and deployed scalable ML pipelines using Python and TensorFlow, directly applicable to Zone IT Solutions's AI infrastructure and data science requirements.

Reason: *Aligned language with Zone IT Solutions's emphasis on Python and TensorFlow in their job description.*

Bullet 2

Original: Reduced model inference latency by 40% through optimization and model quantization.

Modified: Optimized model inference latency by 40% through quantization and performance tuning, supporting production-grade AI systems aligned with Zone IT Solutions's deployment needs.

Reason: *Highlighted production optimization experience to match the company's focus on scalable, production-ready AI solutions.*

Highlighted Skills: Python, TensorFlow, Machine Learning, SQL, Data Science

6. Ethics Reflection

Automated job search agents that interact with hiring processes carry significant ethical responsibilities. This section reflects on the key ethical considerations in this system.

6.1 Bias in Filtering and Ranking

The filtering and ranking logic, while objective in design, can embed systematic bias. The FAANG company exclusion blacklist is a static list requiring manual maintenance — companies that grow to FAANG scale after the list was written would not be excluded. More critically, the skill keyword matching approach favors jobs that use the exact same vocabulary as the candidate's listed skills. A job requiring the same capabilities but using different terminology would be underranked, potentially causing the agent to miss excellent opportunities or unfairly deprioritize employers who write job descriptions differently.

The location preference weighting (+5 points) creates a strong geographic bias. Candidates who receive lower scores for non-preferred-location jobs may miss high-quality opportunities simply because of geography. In a fair system, location should be a filter toggle controlled by the user rather than a scoring penalty.

6.2 Resume Tailoring Fairness

LLM-based resume tailoring raises concerns about authenticity and equity. The system rewrites the candidate's own words to mirror employer language — this is standard practice in manual job applications, but when automated at scale, it could create a situation where candidates with access to AI tools have a systematic advantage over those without. Additionally, the base resume used (John Doe) is a generic profile that does not reflect the diversity of real job seekers. Candidates with non-traditional backgrounds, career gaps, or non-Western names may find that LLM-tailored language inadvertently reinforces majority norms.

6.3 Recommendations

- Audit filtering and ranking criteria regularly for disparate impact across candidate demographics
- Make location preference a hard filter toggle rather than a scoring advantage to avoid geographic penalty
- Test LLM tailoring outputs across diverse candidate profiles to detect language bias patterns
- Be transparent with job seekers that their resume is being automatically modified by AI

7. Links:

- **GitHub Link:** https://github.com/sekhar298/AI-Assignment.git
- **YouTube link:** https://www.youtube.com/watch?v=bmB43EuMC54